



TRAINING HANDBOOK · DATA ENGINEERING TRACK

Databricks & Azure Data Engineering Essentials

A hands-on handbook across the modern Azure lakehouse stack — the Databricks workspace and collaboration, Azure data architectures, Databricks Jobs automation, and Pandas & Python for ETL and analysis. Threaded end-to-end through the **SkyOps Analytics** aircraft-fleet case study.

4

Modules

12

Topics

4

Hands-on labs

2026

Current tooling

What you will learn

Four modules that build a working picture of data engineering on the Azure lakehouse — each with detailed notes, diagrams, illustrative UI walkthroughs, a real-time SkyOps example, runnable code and a hands-on lab.

1 The Databricks Workspace & Collaboration

M1

Workspace features · collaboration & version control · collaborative notebook lab

2 Azure Data Architectures

M2

Architecture patterns in Azure · integration with Azure Data Services

3 Databricks Jobs & Automation

M3

Jobs and their role in automation · scheduling & managing jobs

4 Pandas & Python for ETL & Analysis

M4

Pandas ETL · summary statistics · NumPy numerical analysis · visualization

All twelve requested topics are covered across these four modules. Tooling reflects the 2026 landscape: Unity Catalog, Lakeflow Jobs, Git folders, and Microsoft Fabric / OneLake positioning.

01

PLATFORM

The Databricks Workspace & Collaboration

What the workspace gives you, how teams work together in real time, and how Git brings disciplined version control to notebooks.

The Databricks Workspace & Collaboration

Databricks is the unified analytics platform our **SkyOps Analytics** team uses to turn raw aircraft-fleet telemetry into operational insight. Before building pipelines, you need to know your way around the workspace and how a team collaborates safely inside it.

Learning objectives

- Describe what the Databricks workspace is and its main objects
- Distinguish the control plane from the compute plane
- Use notebooks, compute and Unity Catalog effectively
- Collaborate in real time and leave review comments
- Apply Git-based version control with Git folders
- Run a collaborative, versioned notebook workflow

1.1 What the Databricks workspace is

The **workspace** is the web environment where you write notebooks, run queries, manage data and schedule jobs. On Azure it is deployed as the *Azure Databricks* service inside your subscription. Architecturally it splits into two planes. The **control plane** is managed by Databricks and hosts the UI, notebooks, job scheduler and APIs. The **compute plane** runs the actual Spark clusters on virtual machines in *your* cloud account, next to your data. Your data never leaves your storage — Databricks orchestrates compute against it.

Azure Databricks — Control Plane vs Compute, governed by Unity Catalog

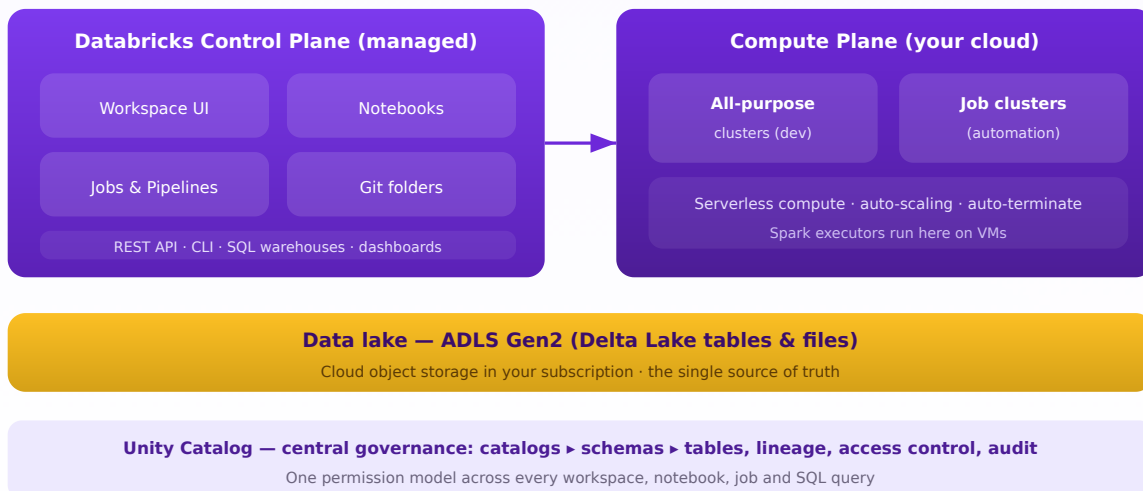


Figure 1.1. Azure Databricks separates the managed control plane from compute that runs in your own cloud, with all data in ADLS Gen2 governed centrally by Unity Catalog.

🔗 Three things every workspace gives you

Compute — clusters and SQL warehouses that run your code. **Workspace objects** — notebooks, dashboards, queries, ML experiments and Git folders, organised in folders. **Unity Catalog** — one governance layer over all data, so a table secured once is secured everywhere.

1.2 Notebooks, compute and the catalog

A **notebook** is an ordered set of cells. Each cell runs code or Markdown, and you can mix languages in one notebook using magic commands: `%sql`, `%md`, `%python`, `%run` (call another notebook) and `%pip` (install a library). A notebook must be **attached to compute** to run.

💡 All-purpose vs job clusters

Use an **all-purpose cluster** (or serverless) for interactive development — several people can attach to it. Use a **job cluster** for scheduled automation: it spins up for the run and terminates after, so you never pay for idle compute. We return to this in Module 3.

Data is addressed through Unity Catalog's three-level namespace `catalog.schema.table` — for us, `skyops_prod.fleet.telemetry`. This replaces ad-hoc paths and the legacy Hive metastore, and is what makes governance and lineage consistent across every workspace, notebook and SQL query.

Name	Type	Last edit
01_ingest_telemetry	Notebook (.ipynb)	2h ago · vivek
02_build_fleet_kpis	Notebook	1d ago · A. Rao
git: skyops-pipelines	Git folder ▶ main	synced ✓
fleet_overview	Dashboard	3d ago

Screenshot 1.1. The Databricks workspace shell — left sidebar for Workspace, Catalog, Jobs & Pipelines, Compute and SQL; the main pane lists notebooks, Git folders and dashboards.

```
# A notebook cell – read a governed Delta table and peek at it
df = spark.table("skyops_prod.fleet.telemetry")
print("rows:", df.count(), "cols:", len(df.columns))
display(df.limit(5)) # display() renders a rich, sortable grid
```

1.3 Collaboration: real-time co-authoring & comments

Multiple people can open the same notebook and edit it **simultaneously**, seeing each other's cursors live — much like a shared document. You can highlight a cell and leave a **comment** to ask a question or request a change, and teammates reply inline. Every notebook also keeps an automatic **revision history**: Databricks snapshots the notebook as you work, so you can view earlier versions and restore one without any Git setup.

⚠️ Auto-history is convenient, but it is not version control

Revision history lives inside one notebook and one workspace. It will not branch, it will not let two features evolve in parallel, and it is not a reviewable record across your whole project. For real engineering discipline you need Git.

1.4 Version control with Git folders

A **Git folder** (formerly called a *Repo*) is a workspace folder linked to a remote repository on GitHub, Azure DevOps, GitLab or Bitbucket. Inside it you get the full Git workflow: create a **branch**, make changes, **commit**, **push**, and open a **pull request** for review before merging to `main`. Each user works in a Git folder under `/Workspace/Users/<you>/` on their own branch.

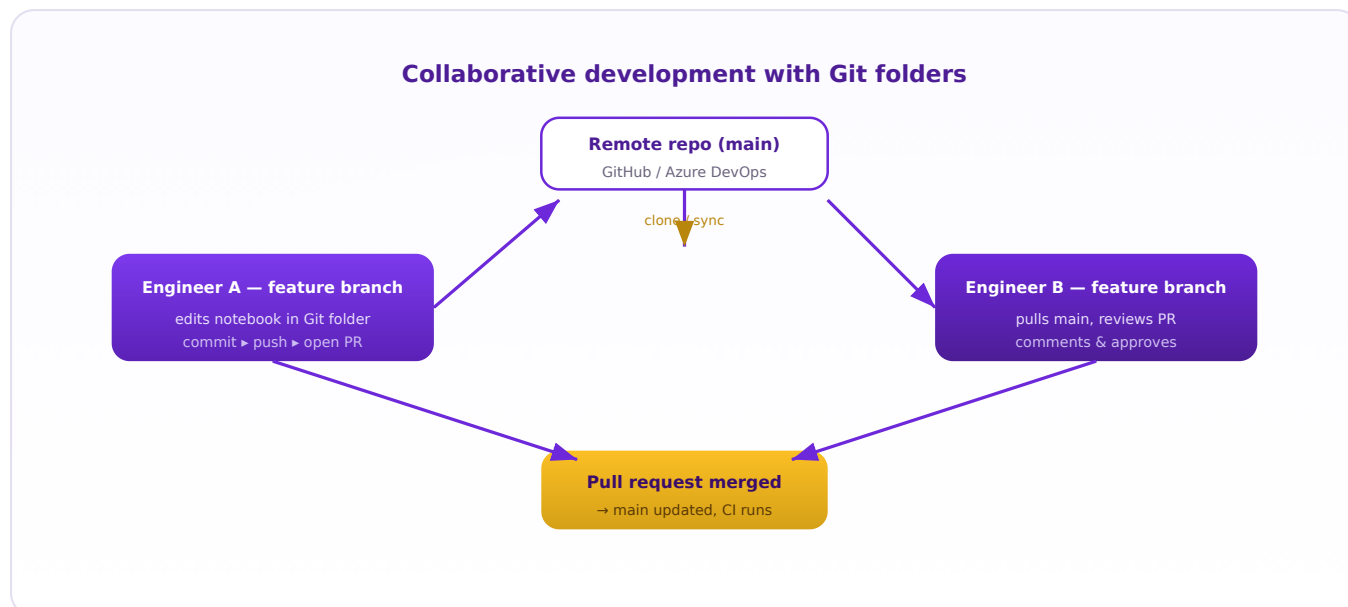
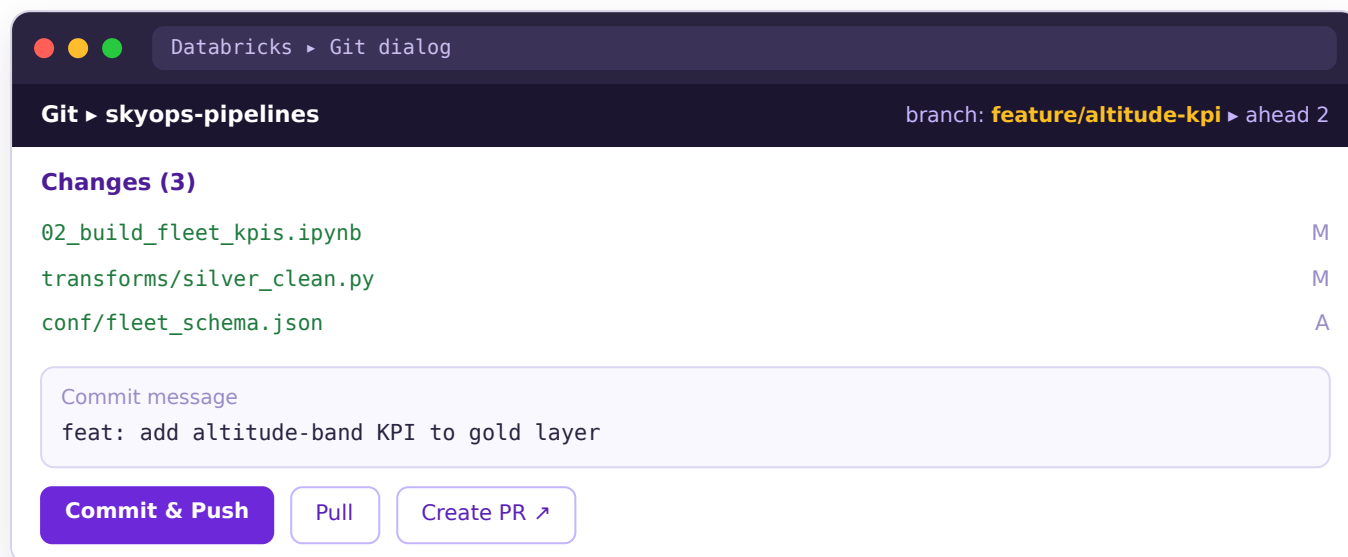


Figure 1.2. A standard branch-based Git workflow inside Databricks: engineers work on feature branches in Git folders, then merge to main through reviewed pull requests.



Screenshot 1.2. The Git panel inside a notebook: staged changes, branch indicator, commit message, and one-click Commit & Push or Create PR — standard version control without leaving the workspace.

💡 Commit notebooks as .ipynb

The Jupyter `.ipynb` format preserves cell outputs, dashboards and visualisations in Git, where the older Databricks source format does not. Beware: a `git pull` that rewrites a notebook resets its live state (outputs, widgets), so commit before pulling.

For production, an admin creates a dedicated **production Git folder** that only automation updates when a PR merges. For full CI/CD, define jobs and pipelines as code with **Databricks Asset Bundles** (YAML) and deploy them through GitHub Actions or Azure Pipelines — the whole project, including the schedule, lives in version control.

SHELL

```
# Typical day, from the terminal or the Git panel
git checkout -b feature/altitude-kpi      # branch off main
# ...edit 02_build_fleet_kpis in the workspace...
git add 02_build_fleet_kpis.ipynb
git commit -m "feat: add altitude-band KPI to gold layer"
git push origin feature/altitude-kpi      # then open a PR for review
```

Lab 1.1 — Collaborative notebook development & versioning

In pairs: (1) create a Git folder from a shared repo and branch off `main`; (2) Partner A adds a cell that reads `skyops_prod.fleet.telemetry` and leaves a comment asking Partner B to add a filter; (3) Partner B edits live, resolves the comment, commits as `.ipynb` and pushes; (4) open a pull request, review each other's diff, merge to `main`; (5) open the notebook's revision history and compare it with the Git commit log. Note which one your team would trust for an audit.

Key takeaways

- The workspace splits a managed control plane from compute that runs in your own cloud, beside your data
- Notebooks mix languages via magics and must attach to compute; data is addressed as `catalog.schema.table`
- Real-time co-authoring and comments speed teamwork; auto revision history is a safety net, not version control
- Git folders bring branches, commits, PRs and review into the workspace — commit notebooks as `.ipynb`
- Asset Bundles version entire jobs and pipelines for proper CI/CD

02

ARCHITECTURE

Azure Data Architectures

The patterns that shape a modern data platform on Azure, and how Databricks plugs into the surrounding Azure data services.

Azure Data Architectures

A platform is only as good as its architecture. This module covers the data-architecture patterns you will meet on Azure, then maps the Azure services that surround Databricks so you know what connects to what — and why.

Learning objectives

- Compare data lake, data warehouse and lakehouse
- Explain the medallion (Bronze/Silver/Gold) pattern
- Recognise lambda, kappa and modern-DW patterns
- Identify the core Azure data services and their roles
- Connect Databricks to ADLS Gen2 securely
- Place Microsoft Fabric and OneLake in the landscape

2.1 Lake, warehouse, lakehouse

A **data lake** stores raw files of any shape cheaply (great flexibility, weak guarantees). A **data warehouse** stores clean, structured, query-optimised tables (strong guarantees, less flexible, costlier). The **lakehouse** unifies both: open files in cloud storage plus a transactional table layer — on Azure that layer is **Delta Lake** on **ADLS Gen2**. You get the lake's economics and openness with the warehouse's ACID transactions, schema enforcement and performance.

Why Delta makes the lakehouse work

Delta Lake adds **ACID transactions, schema enforcement/evolution, time travel** (query an earlier version) and efficient upserts (`MERGE`) on top of Parquet files. That reliability is what lets a plain object store behave like a warehouse.

2.2 The medallion pattern

The dominant lakehouse pattern refines data in three layers. **Bronze** holds raw, as-ingested data (full history, append-only). **Silver** holds cleaned, de-duplicated, conformed and joined entities. **Gold** holds curated, aggregated tables — KPIs and ML features — ready to serve BI and models. Each hop is a notebook or job writing a Delta table.

Medallion architecture on the Azure Lakehouse (Delta Lake)

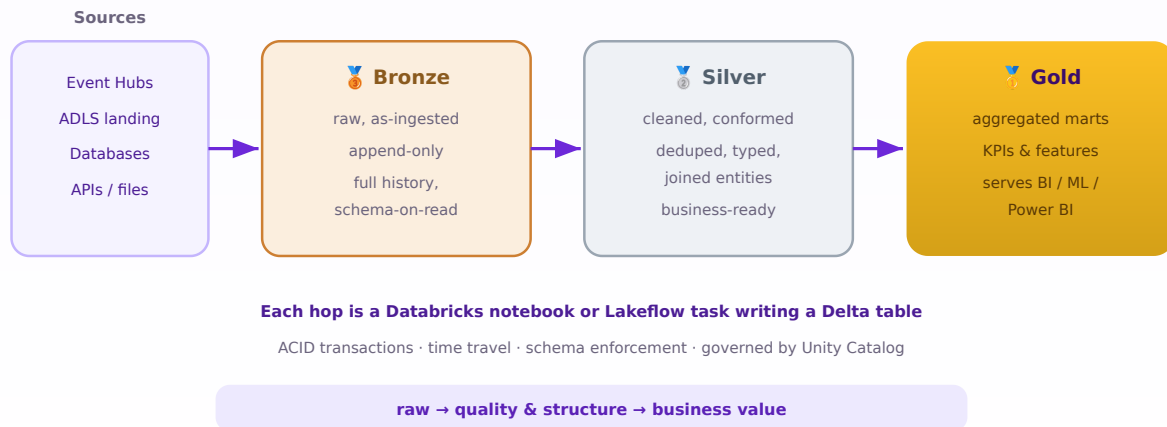


Figure 2.1. The medallion pattern refines data in stages — Bronze (raw) to Silver (clean) to Gold (curated) — each stored as a Delta table on the lakehouse.

✈ SkyOps in medallion terms

Raw sensor messages land in `bronze.telemetry_raw`. We clean units, drop duplicate readings and join the aircraft registry to get `silver.telemetry`. A daily roll-up of fuel burn and altitude bands per tail number becomes `gold.fleet_kpis` — the table Power BI and the data scientists read.

2.3 Lambda, kappa and the modern data warehouse

When low latency matters you will meet two patterns. **Lambda** runs a batch path and a separate speed (streaming) path, then merges them — robust but two codebases to maintain. **Kappa** treats *everything* as a stream and replays history when needed, so there is one pipeline; Spark Structured Streaming and Delta make this practical. The classic **modern data warehouse** pattern — ingest with a pipeline, land in the lake, transform, then load a serving warehouse for BI — still appears widely, increasingly with the lakehouse itself as the serving layer.

💡 Choosing a pattern

Prefer the **medallion lakehouse** as your default. Add a **kappa** streaming path only where freshness is a real requirement (alerting, live dashboards). Reserve a separate serving warehouse for cases with heavy, concurrent SQL where a tuned engine earns its cost.

2.4 Integration with Azure data services

Databricks rarely works alone. Around it sit a small set of Azure services you should recognise:

Service	Role in the platform
ADLS Gen2	The lake — hierarchical object storage holding all Bronze/Silver/Gold Delta data
Azure Data Factory	Cloud ingestion & orchestration; can copy data and trigger Databricks jobs
Azure Event Hubs	Managed event/stream ingestion (Kafka-compatible) feeding the Bronze layer
Azure Key Vault	Stores secrets; surfaced to Databricks via a secret scope (never hard-code keys)
Microsoft Entra ID	Identity (formerly Azure AD) — users, groups and service principals, RBAC
Synapse / Fabric	Warehousing & BI serving; Microsoft Purview adds catalog & lineage
Power BI	The reporting layer, querying Gold tables or a serving warehouse

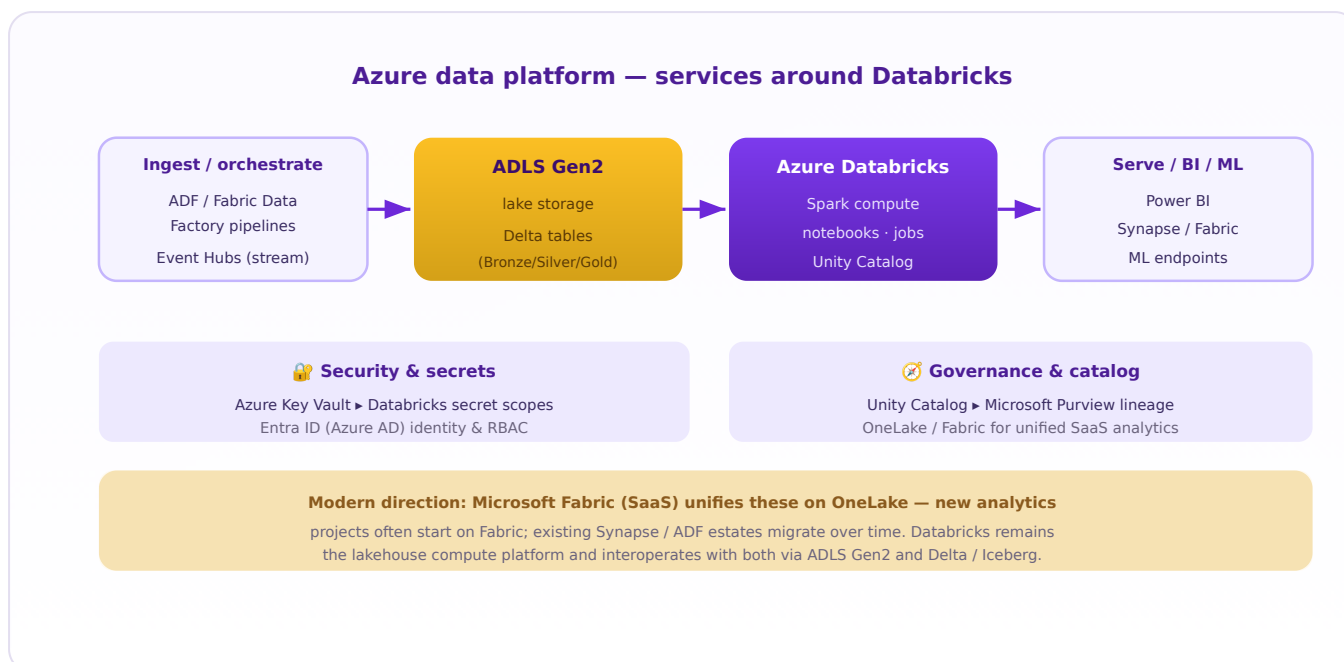


Figure 2.2. Azure data services arranged around Databricks: ingestion and orchestration, ADLS Gen2 storage, Spark compute, and the serving layer, with security and governance cross-cutting.

The clean, modern way to reach the lake is a Unity Catalog **external location** backed by a managed identity, so access is governed centrally. Older material mounts storage or reads the `abfss://` path directly with a key pulled from Key Vault — useful to recognise, but prefer governed access.

● ● ●
PYSPARK / PYTHON

```

# Read a secret from a Key Vault-backed scope (never hard-code credentials)
key = dbutils.secrets.get(scope="skyops-kv", key="adls-sas")

# Preferred: a governed Unity Catalog external location / Volume
df = spark.read.format("delta").load(
    "abfss://lake@skyopsadls.dfs.core.windows.net/bronze/telemetry")
print(df.count())

```

CONSOLE OUTPUT

```

12483910

```

2.5 Where Microsoft Fabric fits in 2026

Microsoft Fabric is Microsoft's strategic, SaaS analytics platform. It unifies data engineering, warehousing, real-time analytics and Power BI over a single managed lake called **OneLake** (Delta format on ADLS Gen2), with **Direct Lake** giving Power BI near-real-time reads. New Microsoft data-platform capabilities are being built on Fabric rather than Synapse.

Synapse, ADF and Databricks alongside Fabric

Azure Synapse and Azure Data Factory are **not deprecated**, but they are in maintenance-and-convergence mode — new projects increasingly start on Fabric, and existing estates migrate over time. **Databricks remains the lakehouse compute platform** and interoperates with both, since everything shares open Delta / Iceberg data on ADLS Gen2. Choose based on existing investment and whether you want SaaS simplicity (Fabric) or open lakehouse control (Databricks).

Lab 2.1 — Design the SkyOps architecture

On a whiteboard, draw the SkyOps platform end to end: name the source (Event Hubs), the lake (ADLS Gen2), the three medallion layers as Delta tables, the compute (Databricks), where secrets come from (Key Vault), and the serving layer (Power BI on Gold). Mark one place you would add a kappa streaming path and justify it in one sentence.

Key takeaways

- The lakehouse unifies lake economics with warehouse guarantees, via Delta Lake on ADLS Gen2
- Medallion (Bronze→Silver→Gold) is the default refinement pattern; each layer is a Delta table
- Lambda/kappa address latency; prefer kappa or a streaming path only where freshness truly matters
- Know the Azure cast: ADLS Gen2, ADF, Event Hubs, Key Vault, Entra ID, Synapse/Fabric, Power BI
- Reach the lake through governed Unity Catalog locations and Key Vault-backed secrets — never hard-code keys
- Microsoft Fabric (OneLake) is the strategic SaaS direction; Databricks remains the open lakehouse compute layer

03

AUTOMATION

Databricks Jobs & Automation

Turning notebooks into reliable, scheduled pipelines with Lakeflow Jobs — tasks, triggers, compute, retries and monitoring.

Databricks Jobs & Automation

A notebook you run by hand is a prototype. A notebook that runs itself every night, recovers from failure and tells you when something breaks is a **pipeline**. Databricks Jobs are how you make that leap.

Learning objectives

- Explain what a Databricks Job is and its role in automation
- Define the job, task and trigger model
- Build a multi-task DAG with dependencies
- Choose job compute over all-purpose compute
- Schedule jobs and add file-arrival triggers
- Manage runs: retries, alerts, monitoring and repair

3.1 What a Databricks Job is

The orchestration product is **Lakeflow Jobs** (you will still see it called *Jobs* or *Workflows*, under **Jobs & Pipelines** in the sidebar). There are three concepts. A **Job** is the unit you schedule and run. It contains one or more **Tasks**, arranged as a **Directed Acyclic Graph (DAG)** so dependencies are explicit. A **Trigger** decides when the job runs. That is the whole model — everything else is configuration.

Tasks come in many types

A task can run a **notebook**, a **Python script or wheel**, **SQL**, a **Lakeflow Declarative Pipeline** (formerly Delta Live Tables), a **dbt** project, or a **dashboard** refresh. Control-flow tasks add **if/else** branching and **for-each** looping, so a single job can express real logic.

3.2 Building a multi-task DAG

Our nightly SkyOps pipeline is one job with four tasks: ingest raw telemetry to Bronze, clean to Silver, build the Gold KPI table, then refresh the dashboard. Each task **depends on** the previous one, so Databricks runs them in order and stops the chain if an upstream task fails.

A Lakeflow Job — tasks as a DAG, with trigger, compute & alerts

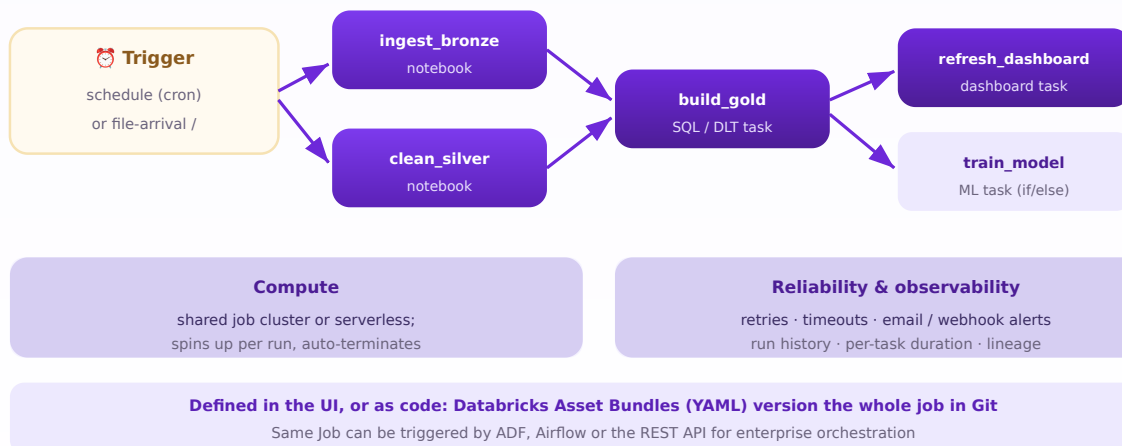


Figure 3.1. A Lakeflow Job models work as a DAG of tasks with dependencies, started by a trigger, run on managed compute, with built-in retries, alerts and run history.

Jobs & Pipelines ► **skyops_nightly_etl** Schedule: 0 0 2 * * ? (02:00 daily) ► Paused: no

Recent runs

Run	Start	Duration	Status
#1043	02:00 · today	6m 12s	Succeeded
#1042	02:00 · yesterday	6m 02s	Succeeded
#1041	02:00 · 2d ago	—	Failed · retried ✓

Retries: 2 · Timeout: 30m · Alerts: email on failure → data-eng@skyops

Task graph

```

graph TD
    ingest_bronze[ingest_bronze ✓] --> clean_silver[clean_silver ✓]
    clean_silver --> build_gold[build_gold ✓]
    build_gold --> refresh_dashboard[refresh_dashboard ✓]
  
```

Screenshot 3.1. A Lakeflow Job: the run history with status and durations on the left (note the auto-retried failure), the task DAG on the right, and the cron schedule in the header.

💡 Use job compute, not an all-purpose cluster

For scheduled work, let each run create its own **job cluster** (or use **serverless**). It starts for the run and terminates after — cheaper and cleaner than leaving an all-purpose cluster running. Tasks in one job can even share a single job cluster to avoid repeated startup.

3.3 Defining a job as code

You can build a job entirely in the UI, but for production it should live in Git. **Databricks Asset Bundles** describe the job — tasks, dependencies, schedule and compute — as YAML you version and deploy through CI/CD.

```
# databricks.yml – a Lakeflow Job as a versioned Asset Bundle (excerpt)
```

```
resources:
  jobs:
    skyops_nightly_etl:
      name: skyops_nightly_etl
      schedule:
        quartz_cron_expression: "0 0 2 * * ?" # 02:00 every day
        timezone_id: "Australia/Sydney"
      email_notifications:
        on_failure: ["data-eng@skyops.example"]
      tasks:
        - task_key: ingest_bronze
          notebook_task: { notebook_path: ./01_ingest_telemetry }
        - task_key: clean_silver
          depends_on: [{ task_key: ingest_bronze }]
          notebook_task: { notebook_path: ./02_build_silver }
        - task_key: build_gold
          depends_on: [{ task_key: clean_silver }]
          notebook_task: { notebook_path: ./03_build_fleet_kpis }
```

3.4 Scheduling & triggers

The most common trigger is a **schedule** written as a Quartz cron expression — `0 0 2 * * ?` means 02:00 daily. Beyond the clock you can use a **file-arrival trigger** (run when new data lands in a storage location), a **table-update trigger** (run when an upstream Delta table changes), or **continuous** mode for an always-on pipeline. You can also start a job on demand from the UI, the REST API, the CLI, ADF or Airflow.

Event-driven beats over-polling

Instead of scheduling the SkyOps ingest every 15 minutes and usually finding nothing, a **file-arrival trigger** fires the job only when a new telemetry batch lands in ADLS — fresher data and less wasted compute.

3.5 Managing & monitoring runs

Reliability comes from a few settings. Add **retries** so a transient failure self-heals. Set a **timeout** so a hung task cannot run forever. Configure **notifications** (email or webhook to Slack/Teams) on failure or when a run runs long. Cap **concurrent runs** so a slow night does not overlap itself.

The **runs** list is your operations dashboard: start time, duration, status and per-task detail, with a visual DAG showing exactly where a failure occurred. When something fails midway, use **Repair run** to re-run only the failed task and its descendants — no need to repeat the successful upstream work.

Make tasks idempotent

Because runs retry and repair, a task should produce the same result if run twice. Prefer `MERGE` or overwrite-by-partition over blind `append`, so a retry never double-writes the same telemetry batch.

Lab 3.1 — Schedule the SkyOps pipeline

Create a job `skyops_nightly_etl` with two notebook tasks where `clean_silver` depends on `ingest_bronze`. Set it to job compute, add a 2 AM schedule, 2 retries, a 30-minute timeout and an email alert on failure. Run it once, then open the runs list and the task DAG. Force the second task to fail, then use **Repair run** to re-run just that task.

Key takeaways

- A Job schedules and runs Tasks arranged as a DAG; a Trigger decides when
- Tasks can run notebooks, SQL, pipelines, Python, dbt, dashboards, plus if/else and for-each control flow
- Prefer job or serverless compute for automation so clusters terminate after the run
- Version jobs as code with Asset Bundles; schedule with cron, file-arrival or table-update triggers
- Reliability = retries + timeouts + alerts + capped concurrency; Repair run re-runs only what failed
- Design tasks to be idempotent so retries and repairs never corrupt data

04

PYTHON

Pandas & Python for ETL & Analysis

The single-machine data toolkit — Pandas for ETL, summary statistics, NumPy for numerical work, and visualization with Python.

Pandas & Python for ETL & Analysis

Not every job needs a cluster. For datasets that fit in memory — a sample, a Gold extract, a daily file — **Pandas** and **NumPy** are the fastest way to clean, analyse and visualise data. They are also the mental model that makes Spark feel familiar, so the skills transfer directly when you scale out.

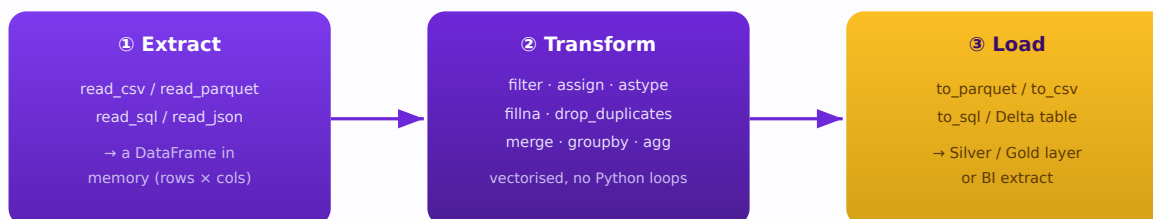
Learning objectives

- Use Pandas to extract, transform and load data
- Clean, combine and reshape DataFrames
- Analyse data with summary statistics
- Apply NumPy for fast numerical analysis
- Explain why vectorised code beats Python loops
- Explore and visualise data with matplotlib & seaborn

4.1 Pandas for ETL

A **DataFrame** is a 2-D table of typed columns (each column is a **Series**). The ETL shape is always the same: **extract** with a reader, **transform** with vectorised operations, **load** with a writer.

Pandas ETL pipeline — Extract ► Transform ► Load



Same mental model as Spark DataFrames — when data outgrows one machine,
lift the identical logic to PySpark / pandas-API-on-Spark with minimal change

Figure 4.1. A Pandas ETL pipeline follows Extract, Transform, Load — the same shape as a Spark pipeline, which makes the concepts transfer directly when you scale out.

```
import pandas as pd

# — Extract —
tel = pd.read_parquet("gold/fleet_telemetry.parquet")
reg = pd.read_csv("ref/aircraft_registry.csv")

# — Transform —
tel = (tel
       .dropna(subset=["altitude_ft"]) # drop unusable rows
       .drop_duplicates(subset=["flight_id", "ts"])
       .astype({"altitude_ft": "int32"})
       .assign(alt_band=lambda d: pd.cut(d.altitude_ft,
                                         bins=[0, 30000, 38000, 99000],
                                         labels=["low", "cruise", "high"])))

fleet = tel.merge(reg, on="aircraft_id", how="left") # join the registry

# — Load —
fleet.to_parquet("silver/fleet_clean.parquet", index=False)
print(fleet.shape)
```

CONSOLE OUTPUT

```
(48217, 11)
```

💡 Prefer vectorised methods over .apply with a Python function

Built-in methods (`.str`, `.dt`, arithmetic, `cut`, `merge`) run in fast C under the hood. Reach for `.apply()` with a Python lambda only when no vectorised equivalent exists — it is the slow path.

4.2 Analysing data with summary statistics

Before modelling anything, **describe** the data. `df.describe()` gives count, mean, std, min, quartiles and max for every numeric column in one call. Combine it with `value_counts()` for categories and `groupby().agg()` to summarise by group.

```
# One-line numeric profile
fleet[["altitude_ft", "fuel_tph"]].describe()

# Categorical frequencies
fleet["type"].value_counts(normalize=True).round(3)

# Grouped summary – the analyst's workhorse
summary = (fleet.groupby("type")
           .agg(flights=("flight_id", "nunique"),
                avg_fuel=("fuel_tph", "mean"),
                p95_alt=("altitude_ft", lambda s: s.quantile(0.95)))
           .round(2)
           .sort_values("avg_fuel", ascending=False))

print(summary)
```

CONSOLE OUTPUT

	flights	avg_fuel	p95_alt
type			
B777	611	7.10	41980.0
A330	902	5.90	41040.0
A350	744	5.40	41550.0
A321	1203	2.70	40870.0
B737	915	2.50	40610.0
A320	1688	2.40	40990.0

Mean, median and the shape of data

The **mean** is sensitive to outliers; the **median** (50% quantile) is not. When they diverge, your distribution is skewed — exactly the kind of thing a histogram (4.4) reveals at a glance.

4.3 NumPy for numerical analysis

Under every Pandas column is a **NumPy array** (`ndarray`) — a contiguous block of one dtype. NumPy lets you compute over whole arrays at once (**vectorisation**), with operations broadcast across elements in compiled C. That is why `a * b` on a million-element array is dramatically faster than a Python loop.

PYSPARK / PYTHON

```
import numpy as np

alt = fleet["altitude_ft"].to_numpy()
fuel = fleet["fuel_tph"].to_numpy()

# Vectorised maths – no loops
fuel_per_1000ft = fuel / (alt / 1000) # element-wise
efficient = np.where(fuel < np.median(fuel), 1, 0) # conditional, vectorised

# Aggregations & statistics
print(np.mean(fuel), np.std(fuel), np.percentile(alt, [50, 95]))

# Broadcasting: subtract each row's type-mean without a loop
type_mean = fleet.groupby("type")["fuel_tph"].transform("mean").to_numpy()
residual = fuel - type_mean # shapes align automatically

# Linear algebra
corr = np.corrcoef(alt, fuel)[0, 1]
print(f"alt/fuel correlation: {corr:.3f}")
```

CONSOLE OUTPUT

```
4.41 1.93 [35010. 41320.]
alt/fuel correlation: -0.118
```

Vectorise, do not loop

If you find yourself writing `for i in range(len(df))`, stop. There is almost always a vectorised Pandas/NumPy expression that is shorter, clearer and 10–100× faster.

4.4 Exploring & visualising data with Python

matplotlib is the foundation; **seaborn** sits on top with attractive statistical charts; and Pandas' own `.plot()` is the quickest route for a fast look. A good exploratory pass pairs a **distribution** view with a **comparison** view.

```
PYSPARK / PYTHON

import matplotlib.pyplot as plt
import seaborn as sns

# Comparison: average fuel burn by aircraft type
ax = summary["avg_fuel"].plot(kind="bar", color="#6d28d9", rot=0)
ax.set_ylabel("tonnes / hour"); ax.set_title("Average fuel burn by aircraft type")
plt.tight_layout(); plt.show()

# Distribution: cruise altitude, with a density curve
sns.histplot(fleet["altitude_ft"], bins=26, kde=True)
plt.title("Distribution of cruise altitude"); plt.show()
```

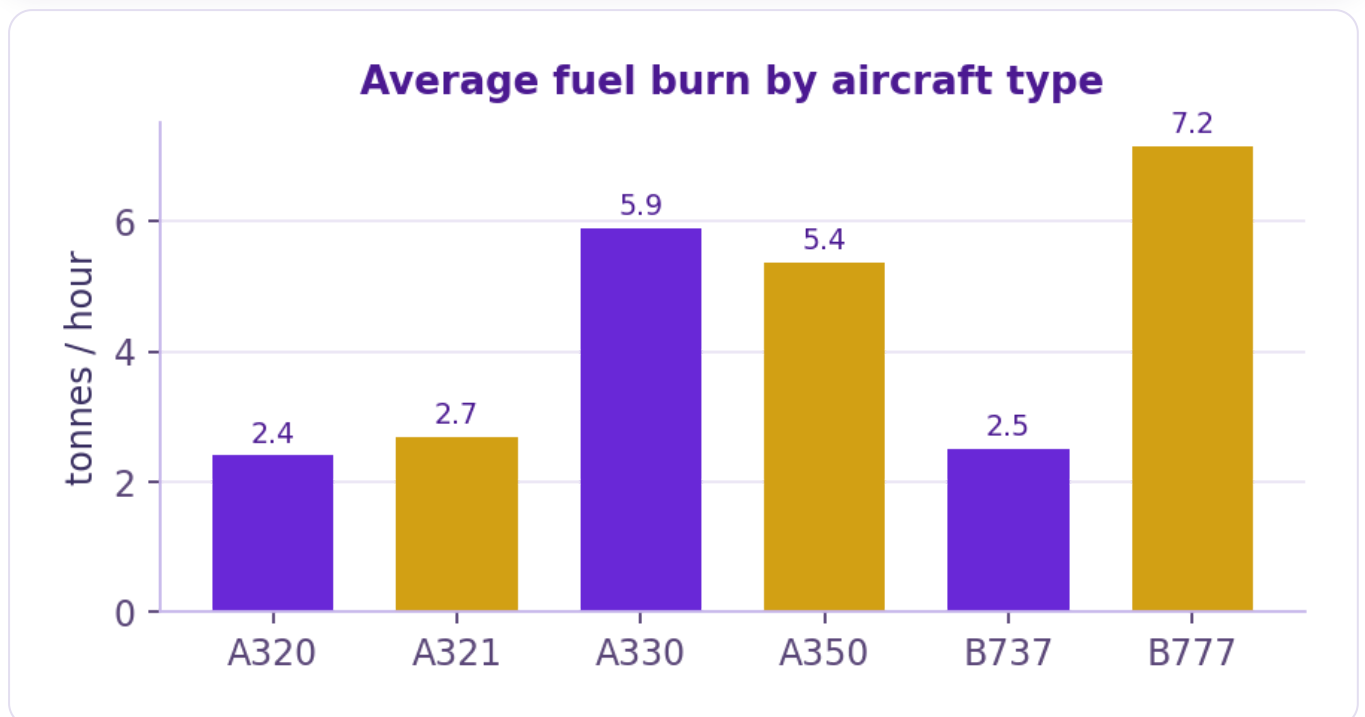


Figure 4.2. A grouped bar chart from `groupby().mean()` — wide-body types (A330, A350, B777) burn far more fuel per hour than narrow-bodies, as expected.

Distribution of cruise altitude

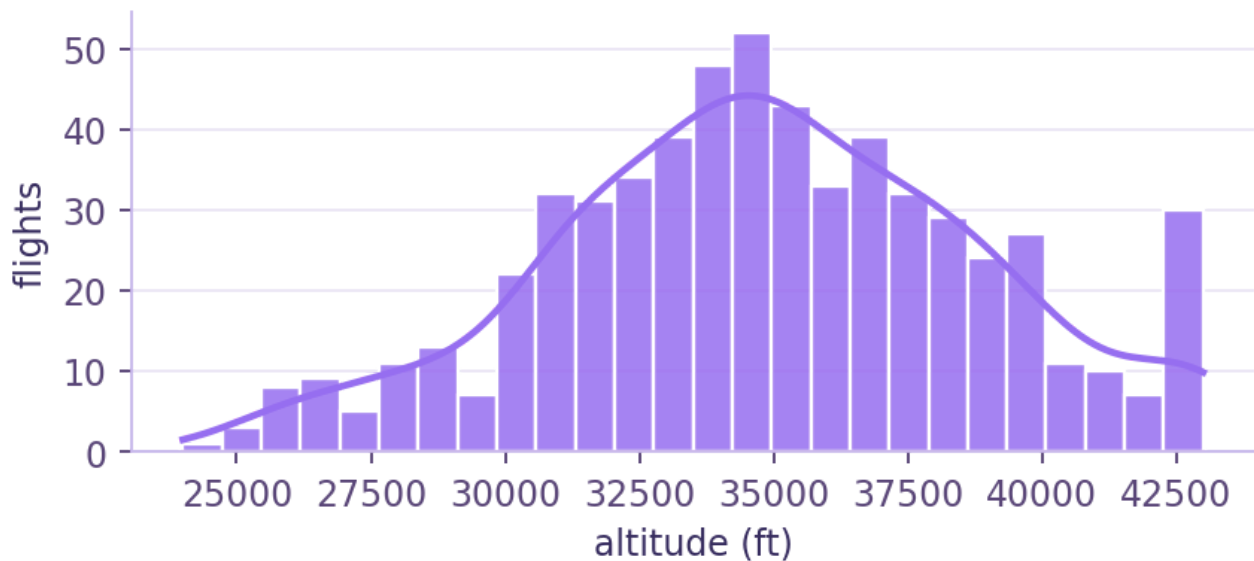


Figure 4.3. A seaborn histogram with density curve — cruise altitude clusters around 35,000 ft with a roughly normal spread, confirming the summary statistics in 4.2.

From chart to decision

Reading these two together, the SkyOps team can target fuel-efficiency analysis at the wide-body fleet and flag flights cruising well below the 35,000 ft mode for closer inspection — a concrete next analysis, straight from exploratory plots.

Lab 4.1 — Explore the fleet extract

Load `silver/fleet_clean.parquet` into Pandas. (1) Print `describe()` and the `type` value counts. (2) Build a grouped summary of average `fuel_tph` and p95 altitude by `alt_band`. (3) Using NumPy, compute the correlation between altitude and fuel. (4) Plot one distribution and one comparison chart, and write a one-sentence finding under each.

Key takeaways

- Pandas ETL is always Extract → Transform → Load; prefer vectorised methods over `.apply`
- `describe()`, `value_counts()` and `groupby().agg()` profile data fast; watch mean vs median for skew
- Every column is a NumPy array — vectorise and broadcast instead of looping for 10–100× speedups
- `matplotlib` + `seaborn` (and `df.plot`) turn statistics into insight; pair a distribution with a comparison
- The Pandas/NumPy model maps directly onto Spark, so these skills scale out unchanged